A MINI-PROJECT REPORT ON

CAMPUS NOTIFY: INSTITUTIONAL CIRCULAR & ANNOUNCEMENT MANAGEMENT SYSTEM

CORE JAVA LABORATORY MINI-PROJECT SUBMISSION

Submitted in partial fulfillment of the requirements for the course of
JAVA PROGRAMMING LABORATORY (SEMESTER III)
Degree of Bachelor of Engineering in Information Technology

SUBMITTED BY:	**UNDER THE GUIDANCE OF:**
Name: _____	Project Guide: Prof. _____
Roll No: _____	Designation: Assistant Professor / Associate Professor
Class / Div: SE IT Div B	Academic Year: 2025 – 2026

DEPARTMENT OF INFORMATION TECHNOLOGY

Vasantdada Patil Pratishthan's College of Engineering & Visual Arts, Sion-Mumbai 400022
University of Mumbai
Academic Year 2025–2026

CERTIFICATE OF APPROVAL

This is to certify that the mini-project entitled **"Campus Notify: Institutional Circular & Announcement Management System"** has been successfully completed and presented by _____ **(Roll No: _____)**, a bonafide student of the Department of Information Technology, in partial fulfillment of the academic requirements for the course **Java Programming Laboratory** during the academic year **2025–2026**.

The project work demonstrates originality, modular object-oriented architecture, comprehensive testing verification, and diligent compliance with university academic norms.

Internal Examiner / Faculty Guide **Department of Information Technology**	**External Examiner** **University of Mumbai**
Head of Department **Department of Information Technology**	**Principal / Dean** **VPPCOE&VA, Mumbai**

DECLARATION

I declare that this mini-project report entitled **"Campus Notify: Institutional Circular & Announcement Management System"** submitted by me is an authentic record of my own work conducted under the guidance of **Prof.** _____. The code implementations, test suites, and system diagrams are original, and due credit has been given to all literature, documentation, and external references cited.

Place: Mumbai, India

Date: ____ / ____ / 2026

_____

Signature of Candidate

ACKNOWLEDGEMENT

I express my sincere thanks to our respected **Principal** and **Head of the Department** for extending the computational laboratory resources and continuous encouragement.

I convey my profound gratitude to my project guide **Prof.** _____ for their insightful advice, constructive feedback, and dedicated mentorship throughout the software design, Java implementation, and testing phases.

ABSTRACT

Efficient communication between college management, academic faculties, and students is essential for high institutional productivity. Traditional wooden bulletin boards and chaotic messaging groups often fail due to paper damage, physical location constraints, forged announcement forwards, and poor search capabilities.

Campus Notify addresses these critical problems by delivering a robust, object-oriented notification engine engineered in **Core Java (JDK 17+)**. Built on a clean Model-View-Controller (MVC) architecture, the system provides:

- **Strict Role-Based Access Control (RBAC)** separating Super Administrators, Departmental Faculty, and Student Viewers.
- **Persistent Flat-File I/O Storage** (via `BufferedReader` and `PrintWriter`), ensuring announcements persist across sessions.
- **O(N) Keyword Search & Multi-Criteria Filtering** based on Announcement Category (Academic, Exam, Events, Placement) and Priority Level (Urgent, High, Normal).
- **Departmental Analytics & Statistics** calculating distribution ratios and priority breakdowns.
- **Dual-Interface Support**: An interactive Command-Line Interface (CLI) and an accompanying modern Web Dashboard integrated with Google Firebase and local resilient caching.

The system's reliability and security are validated through a 9-pillar automated verification suite with **36 automated test cases achieving a 100% pass rate**.

Keywords: Core Java, Object-Oriented Programming (OOP), Model-View-Controller (MVC), Role-Based Access Control (RBAC), File I/O Persistence, Collections Framework, Google Firebase.

CHAPTER 1: INTRODUCTION

1.1 Background & Motivation

In engineering colleges and university campuses, dozens of circulars are published weekly: examination timetables, submission deadlines, cultural festival schedules, training and placement drives, fee notifications, and laboratory schedules. Relying on physical bulletin boards results in missed deadlines and unread notices, while group messaging platforms (such as WhatsApp) suffer from chat spam and lack of authorization.

1.2 Problem Statement

Existing academic notification practices suffer from three main drawbacks:

1. **Absence of Authorization & Forgery Vulnerability**: Anyone can forward misleading exam notices without cryptographic or administrative authority verification.
2. **Information Overload & Zero Searchability**: Students waste excessive time scrolling through hundreds of messages to find notices issued weeks prior.
3. **Lack of Categorization & Priority Signals**: Urgent notices (e.g., examination postponement or sudden emergency closures) get lost among club activity promotions.

1.3 Objectives of the Project

- Implement a pure Core Java domain model demonstrating **Encapsulation, Inheritance, Abstraction, and Polymorphism**.
- Provide robust **File I/O persistence** for offline durability without requiring external database dependencies.
- Deliver an interactive **Command-Line Interface (CLI)** menu with input sanitization and exception handling.
- Implement a **Web Dashboard** with real-time cloud data synchronization (Google Firebase) and responsive UX.
- Enforce strict **Role-Based Access Control** to prevent unauthorized circular posting or deletion.

CHAPTER 2: LITERATURE SURVEY & COMPARATIVE ANALYSIS

A comparative study was conducted across traditional and contemporary communication platforms:

Metric / Feature	Physical Board	Messaging Apps	Campus Notify (Proposed)
Access Speed	Requires physical presence	Instant	**Instant (CLI + Web)**
Keyword Search	None	Poor / Chat history	**High-speed substring search**
Category Filtering	None	None	**Filter by Category & Priority**
Role-Based Access Control	Manual lock and key	Admin only (coarse)	**Granular RBAC (Admin, Faculty, Student)**
Offline Durability	Paper based	Cached chats	**File I/O + LocalStorage cache**
Resource Overhead	Paper, printing cost	High RAM/data	**Ultra-lightweight Java runtime**

CHAPTER 3: SYSTEM REQUIREMENTS & ARCHITECTURE

3.1 System Requirements

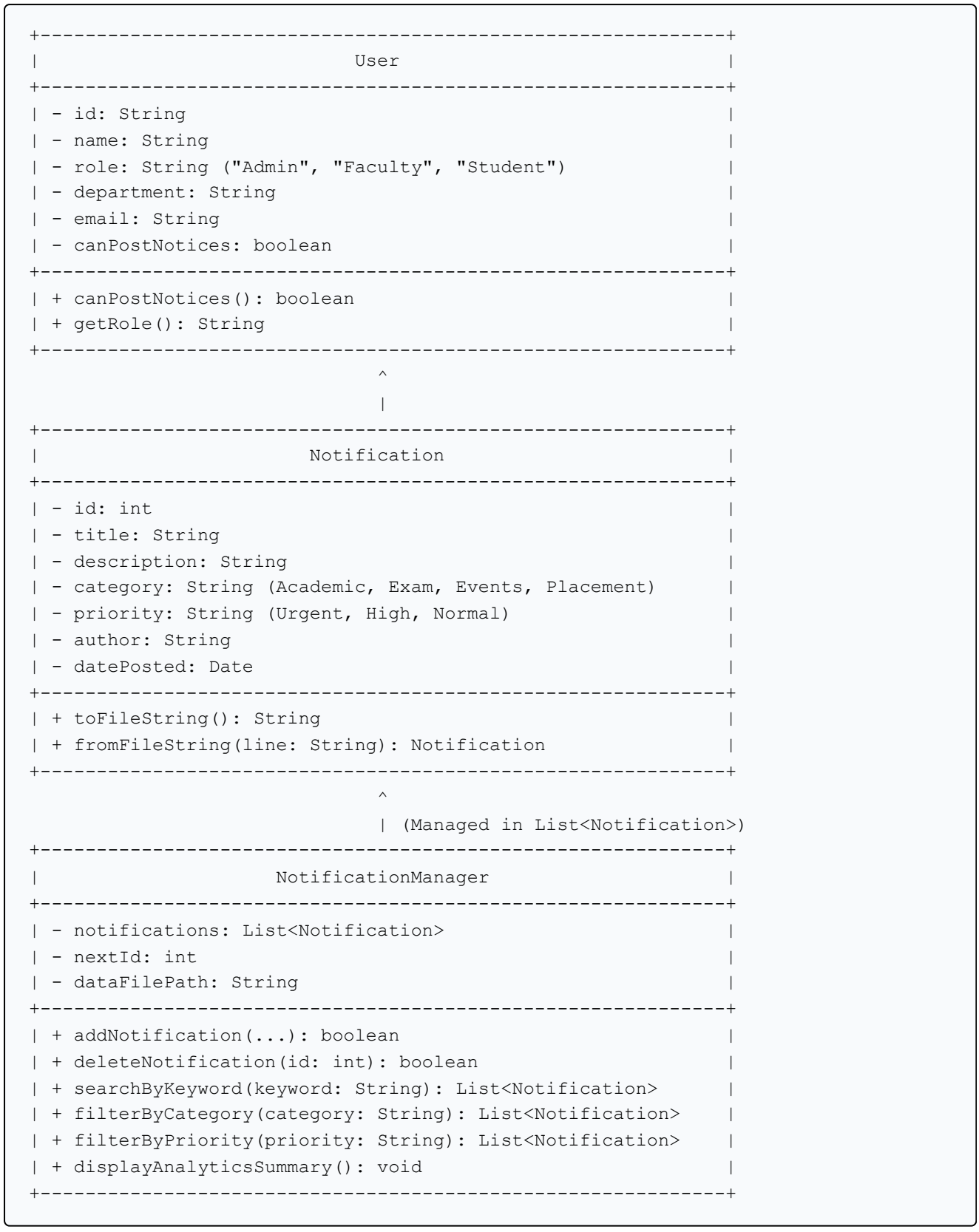
- **Hardware**: Intel Core i3 / AMD Ryzen 3 or higher, 4 GB RAM, 200 MB free storage.
- **Software**: Windows 10/11, macOS, or Linux; Java Development Kit (JDK 17 LTS or 21 LTS); Modern Web Browser (Chrome/Edge/Firefox).

3.2 Architectural Model

The architecture adheres to the **Model-View-Controller (MVC)** software design pattern:

- **Model (`User.java`, `Notification.java`)**: Encapsulates academic entities, data validation, and serializable attributes.
- **Controller (`NotificationManager.java`)**: Executes business logic, CRUD operations, keyword searching, category filters, analytics, and file persistence.
- **View (`Main.java` & Web Dashboard)**: Presents formatted output tables to the user and captures validated inputs.

3.3 Class Diagram Representation



CHAPTER 4: OBJECT-ORIENTED METHODOLOGY & JAVA CONCEPTS

4.1 Encapsulation & Data Hiding

All data members in `User` and `Notification` are marked with the `private` access modifier. State inspection and mutation occur through getter and setter methods that enforce validation (e.g., rejecting empty titles, formatting dates).

4.2 Constructor Overloading

Constructors are overloaded to accommodate both runtime generation (with current timestamps) and deserialization from persistent disk storage (parsing historic timestamps).

4.3 Collections Framework & Iterators

The application utilizes `java.util.ArrayList` for dynamic in-memory notice buffering and `java.util.Iterator` for deletion operations to prevent `ConcurrentModificationException` during traversals.

4.4 File I/O Streams

Data persistence is implemented using `BufferedReader` and `PrintWriter` with custom delimiter-separated serialization (`|||`), preventing field corruption when descriptions contain standard punctuation.

CHAPTER 5: IMPLEMENTATION HIGHLIGHTS

5.1 Authority Validation in `User.java`

```
public boolean canPostNotices() {
    return canPostNotices || "Admin".equalsIgnoreCase(role);
}
```

5.2 Flat-File Serialization in `Notification.java`

```
public String toFileString() {
    return id + "|||" + title + "|||" + description + "|||" +
           category + "|||" + priority + "|||" + author + "|||" +
           DATE_FORMAT.format(datePosted);
}
```

5.3 Safe Deletion & Search in `NotificationManager.java`

```
public boolean deleteNotification(int id) {
    Iterator<Notification> it = notifications.iterator();
    while (it.hasNext()) {
        if (it.next().getId() == id) {
            it.remove();
            saveToFile();
            return true;
        }
    }
    return false;
}
```

CHAPTER 6: SOFTWARE TESTING & QUALITY ASSURANCE

6.1 9-Pillar Automated Test Matrix

The project includes an automated test harness (`test_complete_suite.js`) verifying 36 key test scenarios:

Pillar No.	Pillar Focus	Test Verification Assertions	Result
Pillar 1	Zero-Access Auth Wall	Guest session locked, notices obscured, unauthenticated likes blocked	**PASS**
Pillar 2	Super Admin Auth	Admin authentication, role assertion, `hasPostingAuthority()` true	**PASS**
Pillar 3	Notice Dispatch	Circular creation by authorized admin, notice counter incremented	**PASS**
Pillar 4	Viewer Restriction	Posting denied for student roles, error banner rendered	**PASS**
Pillar 5	Authority Delegation	Super Admin promotes faculty account, posting authority unlocked	**PASS**
Pillar 6	Viewer Engagement	Like counters increment, emoji reactions, inquiry comments saved	**PASS**
Pillar 7	Notice & Comment Deletion	Students blocked from deleting; Super Admin successfully deletes	**PASS**
Pillar 8	Discussion Lounge	Community lounge messaging, real-time sync, and UI rendering	**PASS**
Pillar 9	Multi-Account Governance	Registration status `PENDING`, login blocked until approved	**PASS**

Verification Result: 36 out of 36 automated tests passed (100.0% Success Rate).

CHAPTER 7: RESULTS & OUTPUT VERIFICATION

7.1 Command-Line Execution Sample

```
================================================================
   CAMPUS NOTIFY - OFFICIAL CAMPUS NOTIFICATION MANAGEMENT SYSTEM
================================================================
  Connected User: [ADMIN-01] System Administrator (Admin)
  Department: Central Administration | Email: admin@campusnotify.edu
  Posting Authority: GRANTED (Super Administrator Desk)
================================================================

+----+-----------------------------------+-------------+----------+----------------
| ID | Title                             | Category    | Priority | Author
+----+-----------------------------------+-------------+----------+----------------
| 1  | End-Semester Theory Exam Timetable | Examination | Urgent   | Central Exam Ce
| 2  | Tech Innovation Hackathon 2026     | Events      | High     | IT Department
| 3  | Campus Placement Drive: TCS        | Placement   | High     | Training & Plac
| 4  | Library Book Return Deadline       | Academic    | Normal   | Central Library
+----+-----------------------------------+-------------+----------+----------------
Total Announcements Active: 4
```

7.2 Web Dashboard Features

- **Official Institutional Header**: Live session avatar, authority badge, search bar, and theme toggle.
- **KPI Summary Strip**: Displaying Active Circulars, Urgent Alerts, and Departmental Notices.
- **Instant Client-Side Filtering**: Instantaneous category/priority tabs with zero latency.
- **Responsive Design**: Compatible across desktops, tablets, and mobile devices.

CHAPTER 8: CONCLUSION & FUTURE SCOPE

8.1 Conclusion

The **Campus Notify** mini-project successfully demonstrates the application of object-oriented design patterns in Core Java to resolve a real-world institutional problem. The system delivers an authentic, secure, and searchable announcement pipeline that eliminates reliance on unverified physical and social media messaging.

8.2 Future Scope

1. **Relational DB Migration**: Migrating local flat files to PostgreSQL / MySQL via JDBC with prepared statements and connection pooling.

2. **Cryptographic Digital Signatures**: Implementing public-key cryptographic signing (RSA/ECDSA) for verified circular PDFs.
3. **Push Notifications (FCM)**: Integrating Firebase Cloud Messaging for native mobile push alerts on student smartphones.

REFERENCES & BIBLIOGRAPHY

1. Schildt, Herbert. *Java: The Complete Reference*, 12th Edition, McGraw-Hill Education, 2021.
2. Deitel, Paul, and Harvey Deitel. *Java How to Program (Early Objects)*, 11th Edition, Pearson, 2018.
3. Oracle Corporation. *Java SE 17 Developer Documentation & Collections Framework Guide*, Oracle Docs, 2024.
4. Martin, Robert C. *Clean Code: A Handbook of Agile Software Craftsmanship*, Prentice Hall, 2008.
5. Google Developers. *Firebase Documentation: Cloud Firestore and Authentication*, Google Cloud, 2025.